

CAPCUT Easy CUT

เว็บลากคลิปดิบเข้าไป → ตัดช่วงเงียบ (dead air) → ถอดเสียงทำซับไทยอัตโนมัติ → ส่งออกเป็นโปรเจกต์ CapCut พร้อมตัดต่อ หรือแพ็ก ZIP (วิดีโอ+ซับ) เอนจินเป็น Python
สมองเป็น ffmpeg + faster-whisper + pythainlp

Next.js 14 · React 18

Python engine

ffmpeg · faster-whisper · pythainlp

Portability: STEP 1-4 ✓

Packaging: STEP 5-6 ค้าง

01 ภาพรวม

ระบบทำงานยังไง — มองจากบนสุด

ผู้ใช้คุยกับ เว็บ (Next.js) เท่านั้น เว็บไม่ได้ตัดวิดีโอเอง แต่เป็นตัวส่งงาน — มันเรียก เอนจิน Python ผ่าน API route (spawn python) เอนจินคือคนที่คุยกับ ffmpeg/whisper แล้วเขียนไฟล์โปรเจกต์ CapCut ลงเครื่องโดยตรง งานมี 2 เส้นทางออก และ 1 ชุด AI ช่วยคิดบท

เส้นทาง A

โหมด CapCut draft

เขียน draft เข้าโฟลเดอร์ CapCut
ตรงๆ เปิดมาตัดต่อได้เลย —
ทำงานในเครื่อง user เท่านั้น

เส้นทาง B

โหมด ZIP

แพ็กวิดีโอหลังตัด + ซับ (.srt/.ass) +
transcript เป็น .zip — ขึ้น cloud
ได้

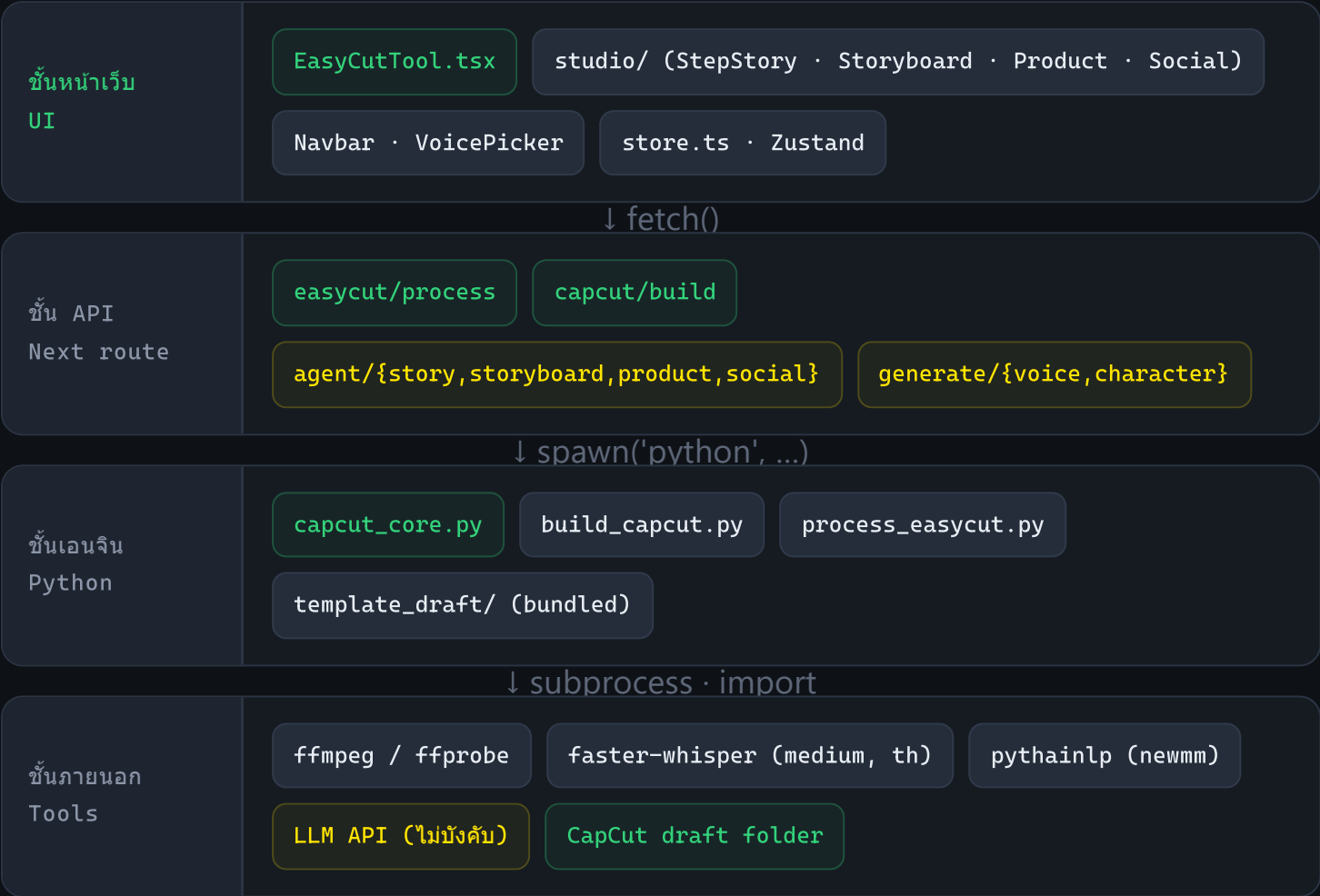
ผู้ช่วย

AI Content Agents

4 เอเจนต์ LLM ช่วยคิดบท/สตอรี่
บอร์ด/ขายของ/แคปชั่นโซเชียล (ไม่
บังคับ)

4 ชั้น จากผู้ใช้งานไปถึง CapCut

แต่ละชั้นคุยกับชั้นถัดลงไปเท่านั้น เว็บไม่แตะ ffmpeg ตรงๆ — ทุกอย่างวิ่งผ่านเอนจิน Python เป็นคอขวดเดียว ทำให้ตรรกะการตัด/ทำซับอยู่ที่เดียวจบ



เส้นทางของคลิป 1 ตัว ตั้งแต่ลากเข้าจนเป็นซับ

นี่คือหัวใจจริงๆ ของระบบ — ทุกชั้นอยู่ใน `capcut_core.py` (1003 บรรทัด, 40 ฟังก์ชัน) ลำดับนี้เป็นลำดับจริง ข้ามขั้นไม่ได้ หน่วยเวลาใน draft CapCut = ไมโครวินาที

01

รับคลิป

ลาก/อัปคลิปหลายไฟล์เข้าเว็บ เรียงตามชื่อ (natural sort)

```
read_clips()
```

02

ตรวจ Dead air

ffmpeg silencedetect หาช่วงเงียบ แล้วคำนวณช่วงที่จะเก็บ

```
detect_silence ·  
compute_keeps
```

03

ตัด + ต่อ

ตัดช่วงเงียบทิ้ง (tighten) แล้วรวมทุกคลิปเป็นไฟล์เดียว

```
tighten_clip ·  
concat_clips
```

04

ถอดเสียง

faster-whisper (ไทย) ถอดเสียงพร้อมเวลาแต่ละคำ (word timestamps)

```
transcribe()
```

05

ซ่อมคำไทย

ต่อขึ้น BPE ที่ whisper หั่นกลางพยางค์กลับ + ตัดคำใหม่ pythainlp + AI ตรวจคำ (ไม่บังคับ)

```
merge_words_to_tokens ·  
llm_thai_corrections
```

06

จัดซับ

วางจังหวะซับให้ตรงเสียง คำสำคัญกลางจอ ไฮไลท์ **เขียว/เหลือง**

```
captions_phrases_high  
light
```

07

ส่งออก

เขียน draft CapCut (JSON) หรือแพ็ก ZIP (mp4+srt/ass)

```
build_draft() / ZIP  
writer
```

ต่างกันที่ปลายทาง — และนี่คือหัวใจของการแจก user

ชั้น 01–06 ใช้ร่วมกันทั้งคู่ ต่างกันแค่ชั้น 07 ปลายทาง ตัวนี้กำหนดว่า feature ไหนชั้น cloud ได้ ไหนต้องอยู่ในเครื่อง user

เส้นทาง A `build_capcut.py`

→ %LOCALAPPDATA%\CapCut\...
 \com.lveditor.draft\<ชื่องาน>

- เขียนไฟล์ draft เข้าโฟลเดอร์ CapCut ตรงๆ
- เปิด CapCut เห็นโปรเจกต์เลย ตัดต่อต่อได้
- ได้ซับสลาย + pop animation + zoom transition
- ชั้น cloud ไม่ได้ — ต้องรันในเครื่องที่มี CapCut

เส้นทาง B `process_easycut.py`

→ CAPCUT_Easy_CUT.zip (mp4 + srt/ass + transcript)

- ตัด dead air + ทำซับ แล้วแพ็กเป็น .zip
- ไม่พึ่ง CapCut ในเครื่อง เอาไปเปิดที่ไหนก็ได้
- เหมาะกับการเป็นเว็บ SaaS (ต้องมี GPU/คิวงาน)
- ไม่ได้โปรเจกต์ CapCut พร้อมตัด (ได้แค่ไฟล์)

4 เอเจนต์ช่วยคิดก่อนถ่าย

ชุดนี้แยกจากสายพานตัดต่อ — เป็น LLM ล้วน สร้าง prompt กลางที่ [agentBuild.ts](#) รันได้ทั้งฝั่ง server และ browser (Puter) รองรับหลายเจ้า: gemini / openai / anthropic / groq / cerebras / openrouter / local

AGENT 1 · Story

คิดบท/เรื่องจากไอเดีย + โทน + ความยาว

AGENT 2 · Storyboard

แตกบทเป็นซีน (voiceover ต่อข้อต่อ)

AGENT 3 · Product

มูมขายของ/จุดขายสินค้า

AGENT 4 · Social

แคปชั่น/โพสต์ต่อแพลตฟอร์ม

บทที่ได้ (script.json) ส่งต่อให้ `build_capcut.py --script` ใช้เป็นซับที่ "ถูกปั๊ม" แล้วจับเวลาจาก whisper แทนการถอดเสียงล้วน

ไฟล์ไหนทำอะไร

ไฟล์	ชั้น	หน้าที่
components/EasyCutTool.tsx	Web	UI ลากคลิกป ตั้งค่า ส่งออก
components/studio/*	Web	Wizard คิดบท → สตอรี่บอร์ด → ข่ายของ → โชว์เซลล์
lib/agentBuild.ts · prompts.ts	Web	สร้าง prompt กลางของ 4 เอเจนต์
lib/store.ts · types.ts	Web	state (Zustand) + โครงข้อมูล
api/easycut/process/route.ts	API	spawn process_easycut.py → คืน .zip
api/capcut/build/route.ts	API	spawn build_capcut.py → สร้าง draft
capcut-auto/capcut_core.py	Engine	เอนจินกลาง: dead air · whisper · ซับ · draft · portability
capcut-auto/build_capcut.py	Engine	entry โหมด CapCut draft
capcut-auto/process_easycut.py	Engine	entry โหมด ZIP
capcut-auto/template_draft/	Engine	template draft สำเร็จรูป (แจกไปกับแอป)

สิ่งที่แก้ไขเพื่อให้รันบนเครื่อง user คนอื่นได้

เดิมเอนจินผูกกับเครื่อง dev (path ส่วนตัว · ต้องมี draft เดิม · โมเดลหนัก · พัง PATH) 4 จุดนี้
แก้ไขระบบ "ย้ายเครื่องได้" — ผ่านการทดสอบจำลองเครื่องเปล่าด้วยคลิปชุดไทยจริง 38 วินาที
แล้ว

STEP 1 · Template

Bundled template + token

เดิม: ต้องมี draft เดิมในเครื่อง ไม่งั้น error

ฝัง `template_draft/` ที่ล้าง path ส่วนตัวออก แทนด้วย
`__LOCALAPPDATA__` แล้ว `_substitute_tokens()`
แปลงเป็น path จริงตอนสร้าง

STEP 2 · Resource

pop/effect path แบบ dynamic

เดิม: hardcode `C:/Users/onpre/...`

`resolve_effect_cache(id)` หา cache ตามเครื่อง user
เอง — ไม่มีคั่น `""` ให้ CapCut โหลดเองด้วย `resource_id` +
ปิดได้ด้วย `pop_anim_on`

STEP 3 · Whisper

โมเดลเบาลง + auto GPU

เดิม: large-v3 บน CPU (~3GB, ซ้ำมาก)

default `medium` · `device=auto` (เจอ GPU ใช้เลย ไม่มี
fallback CPU) · ปรับผ่าน env · มี progress %

STEP 4 · Tools

ffmpeg/font resolve + เช็กติดตั้ง

เดิม: เรียก ffmpeg ลอยๆ พัง PATH · font hardcode

`FFMPEG/FFPROBE` ผ่าน env (เตรียม bundle) ·
`resolve_font()` fallback ·
`ensure_ffmpeg/ensure_capcut()` เช็กก่อนทำงาน

08 ไรต์แอปการแจก

จากโค้ด → ตัวติดตั้งที่ user ทั่วไปกดใช้ได้

✓

STEP 1-4 · Portability (เสร็จ)

engine รันบนเครื่องที่ไม่ใช่ของ dev ได้แล้ว — ทดสอบจำลองเครื่องเปล่าผ่าน

5

PyInstaller → engine.exe

รวม faster-whisper + pythainlp เป็น .exe เดียว — user ไม่ต้องลง Python

6

Bundle ffmpeg + Tauri installer

ฝัง ffmpeg + ห่อ UI เป็นตัวติดตั้ง .exe/.msi ตัวเดียว ดับเบิลคลิกลงได้

▲ **ยังต้องพิสูจน์ด้วยตาจริง:** การเปิด draft ใน CapCut GUI + export คลิป — โครงสร้าง draft ตรงกับที่ใช้งานจริงทุก field (version 360000) แล้ว แต่ยังไม่ได้ยืนยันภาพ render สด และความเสี่ยงระยะยาว: format draft ผูกกับ CapCut เวอร์ชัน — ถ้า CapCut อัปเดตใหญ่อาจต้องอัปเดต template

